INTRO TO DYNAMIC PROGRAMMING: SIMPLE EXAMPLES

Quantitative Economics 2025

Piotr Żoch

December 9, 2025

INTRODUCTION

- Shortest Path
- Tree-cutting
- Resource Extraction

SHORTEST PATH

SHORTEST PATH

- We have a set of nodes, $S = \{s_1, s_2, \ldots, s_N\}$.
- Let $F_s \subseteq S$ be the set of nodes reachable from node s in one step.
- $c(s, s')$ is the cost of moving from node s to node $s' \in F_s$.
- Goal: Find the shortest path from a starting node to destination $d \in S$.

SHORTEST PATH

- Let $J(s)$ be the minimum cost from node s to destination d.
- If we knew $J(s)$ for all $s \in S$, then from starting node s, the optimal next node would satisfy:

$$s' = \underset{s' \in F_s}{\operatorname{argmin}} \underbrace{c(s, s')}_{\text{immediate cost}} + \underbrace{J(s')}_{\text{future cost}} .$$

- Therefore, the minimum cost from s must satisfy:

$$J(s) = \min_{s' \in F_s} c(s, s') + J(s')$$

- This is the Bellman equation for the shortest path problem.
- State variable: current node s.

SHORTEST PATH: SOLUTION METHOD

- Problem: We need to find $J(s)$ for all $s \in S$, but we only know the boundary condition: $J(d) = 0$.
- Solution: Value function iteration
- Initial guess $J_0(s)$ for all $s \in S$:
  - $J_0(d) = 0$ (boundary condition);
  - $J_0(s) = M$ for all $s \neq d$, where M is very large.
- The boundary condition is exact; only $J(s)$ for $s \neq d$ needs to be computed.

SHORTEST PATH: ALGORITHM

- Value function iteration algorithm:
  1. Initialize: Set $k = 0$ and choose initial guess J_0.
  2. Update: Compute $J_{k+1}(s) = \min_{s' \in F_s} c(s, s') + J_k(s')$ for all $s \in S$.
  3. Check convergence: If $J_{k+1} = J_k$, stop. Otherwise, set $k = k + 1$ and return to step 2.
- The algorithm converges to the true minimum cost $J(s)$.

SHORTEST PATH: WHY DOES THIS WORK?

- The Bellman operator T is defined as:

$$(TJ)(s) = \begin{cases} 0 & \text{if } s = d \\ \min_{s' \in F_s} c(s, s') + J(s') & \text{if } s \neq d \end{cases}$$

- Under mild conditions (e.g., all costs $c(s, s') > 0$), T is a contraction mapping.
- The Contraction Mapping Theorem guarantees:
  - Unique fixed point J^* (the true minimum cost function).
  - $J_k \to J^*$ for any initial guess J_0.
- The boundary condition $J(d) = 0$ is part of the fixed point, not a failure of CMT!

TREE-CUTTING

TREE-CUTTING

- You own a tree of current size $s \in \{0, h, 2h, \ldots, \bar{S}\}$.
- Each period you face a binary decision: cut down or wait.
- Cut down now:
  - Receive revenue $f(s)$ dollars immediately.
  - Tree is gone—problem ends (terminal state).
- Wait one period:
  - Tree grows by h units: $s \to s + h$ (unless already at maximum $\bar{S}$).
  - You make the decision again next period.
- Objective: Maximize present discounted value of revenue.
- Discount factor: $\beta = \frac{1}{1+r} \in (0, 1)$, where $r > 0$ is the interest rate.

TREE-CUTTING: BELLMAN EQUATION

- Let $v(s)$ be the value function: maximum discounted revenue starting with tree of size s.
- State variable: tree size $s \in \{0, h, 2h, \ldots, \bar{S}\}$.
- Bellman equation:

$$v(s) = \max\left\{ \underbrace{f(s)}_{\text{cut now}}, \underbrace{\beta \cdot v(\min\{\bar{S}, s+h\})}_{\text{wait}} \right\}.$$

- Optimal policy: $g^*(s) \in \{\text{cut}, \text{wait}\}$

$$g^*(s) = \begin{cases} \text{cut} & \text{if } f(s) \geq \beta \cdot v(\min\{\bar{S}, s+h\}) \\ \text{wait} & \text{otherwise} \end{cases}$$

TREE-CUTTING: INTUITION

- Trade-off: Cut now for $f(s)$ today vs. wait for $\beta \cdot v(s+h)$ in present value.
- Typically, f is increasing and convex (larger trees worth more per unit).
- Optimal policy structure (under natural assumptions):
  - Threshold policy: $\exists$ threshold size s^*.
  - Cut if $s \geq s^*$, wait if $s < s^*$.
- Solve using value function iteration or policy iteration.

TREE-CUTTING: EXTENSIONS

How to modify the Bellman equation?

1. Cutting costs $c > 0$ dollars?
2. Tree might not grow (prob. p)?
3. Tree might die (prob. p, receive $\alpha f(s)$, $\alpha \in (0, 1)$)?
4. Tree can become sick (prob. p); sick trees don't grow; sick trees recover (prob. q)?

TREE-CUTTING: EXTENSION 1 (CUTTING COST)

- Setup: Cutting costs $c > 0$ dollars.
- Bellman equation:

$$v(s) = \max\left\{\underbrace{f(s) - c}_{\text{cut}}, \underbrace{\beta \cdot v(\min\{\bar{S}, s+h\})}_{\text{wait}}\right\}.$$

- Implication: Cutting less attractive $\Rightarrow$ threshold s^* increases (wait longer).

TREE-CUTTING: EXTENSION 2 (GROWTH UNCERTAINTY)

- Setup: With prob. p, tree doesn't grow.
- State transition: $s' = \begin{cases} \min\{\bar{S}, s+h\} & \text{prob. } 1-p \\ s & \text{prob. } p \end{cases}$
- Bellman equation:

$$v(s) = \max\{f(s), \beta \cdot [(1-p) \cdot v(\min\{\bar{S}, s+h\}) + p \cdot v(s)]\}.$$

- Implication: Uncertainty reduces wait value $\Rightarrow$ cut earlier.

TREE-CUTTING: EXTENSION 3 (DEATH RISK)

- Setup: With prob. p, tree dies (forced harvest, receive $\alpha f(s)$, $\alpha \in (0,1)$).
- State transition: $\begin{cases} \text{size } \min\{\bar{S}, s+h\} & \text{prob. } 1-p \\ \text{dead, get } \alpha f(s) & \text{prob. } p \end{cases}$
- Bellman equation:

$$v(s) = \max\{f(s), \beta \cdot [(1-p) \cdot v(\min\{\bar{S}, s+h\}) + p \cdot \alpha f(s)]\}.$$

- Note: $\alpha f(s)$ is terminal payoff.
- Implication: Death risk $\Rightarrow$ cut earlier.

TREE-CUTTING: EXTENSION 4 (HEALTH STATUS)

- Setup: Tree has health. Sick trees don't grow.
- Healthy $\xrightarrow{p}$ Sick; Sick $\xrightarrow{q}$ Healthy.
- State space: $(s, \text{health}) \in \{0, h, \ldots, \bar{S}\} \times \{H, S\}$.
- Two value functions: $v^H(s)$, $v^S(s)$.

TREE-CUTTING: EXTENSION 4 (BELLMAN EQUATIONS)

Healthy tree:

$$v^H(s) = \max\left\{f(s), \beta\left[(1-p)\cdot v^H(\min\{\bar{S}, s+h\}) + p\cdot v^S(s)\right]\right\}$$

Sick tree:

$$v^S(s) = \max\left\{f(s), \beta\left[q\cdot v^H(\min\{\bar{S}, s+h\}) + (1-q)\cdot v^S(s)\right]\right\}$$

- Healthy: stays healthy (grows) w.p. $1-p$, becomes sick w.p. p.
- Sick: recovers (grows) w.p. q, stays sick w.p. $1-q$.
- Result: $v^H(s) \geq v^S(s)$ for all s.

RESOURCE EXTRACTION

RESOURCE EXTRACTION

- You own a non-renewable resource of size $s \in \{0, 1, 2, \ldots, \bar{S}\}$.
- Each period: observe price p_t, choose extraction $x_t \in \{0, 1, \ldots, s_t\}$.
- Price dynamics: $p_t \overset{iid}{\sim} \phi$ with support $\mathcal{P} = \{p_1, p_2, \ldots, p_N\}$.
- Cost function: $c(x)$ is the cost of extracting x units.
- Resource dynamics: $s_{t+1} = s_t - x_t$ (resource depletes, never replenishes).
- Objective: Maximize expected present discounted value of profit:

$$\mathbb{E}_0 \sum_{t=0}^{\infty} \beta^t \left(p_t x_t - c(x_t)\right)$$

RESOURCE EXTRACTION: BELLMAN EQUATION

- State variables: (s, p) where $s \in \{0, 1, \ldots, \bar{S}\}$ is stock, $p \in \mathcal{P}$ is price.
- Bellman equation:

$$v(s,p) = \max_{x \in \{0,1,\ldots,s\}} \left\{ px - c(x) + \beta \sum_{p' \in \mathcal{P}} v\big(s - x, p'\big)\phi\big(p'\big) \right\}.$$

- Boundary condition: $v(0, p) = 0$ for all p.
- Optimal policy: $x^*(s, p)$ gives extraction as function of stock and price.

RESOURCE EXTRACTION: SIMPLIFIED VERSION

- To understand the computation, consider a simplified version with constant price p.
- State variable is just $s \in \{0, 1, 2, \ldots, \bar{S}\}$.
- Bellman equation:

$$v(s) = \max_{x \in \{0,1,\ldots,s\}} \{px - c(x) + \beta v(s - x)\}.$$

- This is a single-dimensional problem in stock s.

COMPUTING THE VALUE FUNCTION: MATRIX REPRESENTATION

- The value function $v(s)$ can be stored as a vector:

$$\mathbf{v} = \begin{pmatrix} v(0) \\ v(1) \\ v(2) \\ \vdots \\ v(\bar{S}) \end{pmatrix}$$

- Each iteration updates this entire vector.
- For each state s, we need to maximize over extraction choices $x \in \{0, 1, \ldots, s\}$.
- Equivalently: choose future stock $s' = s - x \in \{0, 1, \ldots, s\}$.

COMPUTING THE VALUE FUNCTION: TABLE STRUCTURE

- At state s with current guess v_k, compute payoff for each future stock s':

$$\text{payoff}(s') = p(s - s') - c(s - s') + \beta v_k(s')$$

- Create a table: rows = current stock, columns = future stock

Current s / Future s'	0	1	2	$\cdots$
0	$-c(0) + \beta v_k(0)$	—	—	$\cdots$
1	$p - c(1) + \beta v_k(0)$	$-c(0) + \beta v_k(1)$	—	$\cdots$
2	$2p - c(2) + \beta v_k(0)$	$p - c(1) + \beta v_k(1)$	$-c(0) + \beta v_k(2)$	$\cdots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$

- $v_{k+1}(s)$ = max across row s (over feasible $s' \leq s$).

COMPUTING THE VALUE FUNCTION: EXAMPLE WITH $\bar{S} = 3$

- Suppose $\bar{S} = 3$, $p = 10$, $c(x) = 2x$, $\beta = 0.9$.
- At iteration k, suppose $v_k^T = (0, 8, 16, 24)$ (for $s = 0, 1, 2, 3$).
- Payoff table (current profit + discounted future value):

s/s'	0	1	2	3
0	$0 - 0 + 0 = 0$	—	—	—
1	$10 - 2 + 0 = 8$	$0 - 0 + 7.2 = 7.2$	—	—
2	$20 - 4 + 0 = 16$	$10 - 2 + 7.2 = 15.2$	$0 - 0 + 14.4 = 14.4$	—
3	$30 - 6 + 0 = 24$	$20 - 4 + 7.2 = 23.2$	$10 - 2 + 14.4 = 22.4$	$0 - 0 + 21.6 = 21.6$

- $v_{k+1}^T = (0, \max\{8, 7.2\}, \max\{16, 15.2, 14.4\}, \max\{24, 23.2, 22.4, 21.6\}) = (0, 8, 16, 24)$

COMPUTING THE VALUE FUNCTION: ROW-BY-ROW MAXIMIZATION

- For each row $s = 0, 1, 2, \ldots, \bar{S}$:
  1. Compute payoff (reward + continuation) for all feasible future stocks $s' \in \{0, 1, \ldots, s\}$.
  2. Find the maximum across that row: this is $v_{k+1}(s)$.
  3. Record which column achieved the maximum: this is the optimal s'^*.
- After processing all rows, we have:
  - Updated value function: $v_{k+1}(0), v_{k+1}(1), \ldots, v_{k+1}(\bar{S})$.
- Iterate until convergence: $\|\mathbf{v}_{k+1} - \mathbf{v}_k\| < \varepsilon$.

STOCHASTIC PRICES: ADDING THE PRICE DIMENSION

- Return to full problem with i.i.d. prices: $p \overset{iid}{\sim} \phi, \mathcal{P} = \{p_1, \ldots, p_N\}$.
- State space: $(s, p) \in \{0, \ldots, \bar{S}\} \times \mathcal{P}$.
- Need value function for each price: $v(\cdot, p_1), \ldots, v(\cdot, p_N)$.
- Store as matrix ($\bar{S} + 1$ rows, N columns):

$$\mathbf{V} = \begin{pmatrix} v(0, p_1) & \cdots & v(0, p_N) \\ \vdots & \ddots & \vdots \\ v(\bar{S}, p_1) & \cdots & v(\bar{S}, p_N) \end{pmatrix}$$

STOCHASTIC PRICES: MODIFIED BELLMAN EQUATION

- Bellman equation:

$$v(s, p_i) = \max_{s' \in \{0,\ldots,s\}} \left\{ p_i(s - s') - c(s - s') + \beta \sum_{j=1}^{N} v(s', p_j)\phi(p_j) \right\}$$

- Key: Future value is expected over all prices (because prices are i.i.d.).
- Define expected continuation value: $\bar{v}(s') = \sum_{j=1}^{N} v(s', p_j)\phi(p_j)$
- Then:

$$v(s, p_i) = \max_{s'} \left\{ \underbrace{p_i(s - s') - c(s - s')}_{\text{depends on } p_i} + \underbrace{\beta \bar{v}(s')}_{\text{same for all } p_i} \right\}$$

- If prices were not i.i.d., continuation value would depend on current p_i.

STOCHASTIC PRICES: ALGORITHM

For each iteration k:

1. Compute expected continuation: $\bar{v}_k(s') = \sum_{j=1}^{N} v_k(s', p_j)\phi(p_j)$ for all s'.
2. For each (s, p_i): Build payoff table using $\bar{v}_k(s')$, maximize to get $v_{k+1}(s, p_i)$.
3. Check: $\|\mathbf{V}_{k+1} - \mathbf{V}_k\| < \varepsilon$?

Example: $\bar{S} = 3$, $\mathcal{P} = \{p_L, p_H\}$, $\phi(p_L) = 0.3$, $\phi(p_H) = 0.7$.

- Compute: $\bar{v}_k(s) = 0.3 \cdot v_k(s, p_L) + 0.7 \cdot v_k(s, p_H)$ for $s = 0, 1, 2, 3$.
- Build two payoff tables: one for p_L, one for p_H (both use same $\bar{v}_k$).
- Get: $v_{k+1}(\cdot, p_L)$ and $v_{k+1}(\cdot, p_H)$.